

JavaScript Study Notes

Chapters 1-5: Alerts, Variables & Math Expressions — Daniyal Warsi

Chapter 1: Alerts

What is an Alert?

An alert is a pop-up box that displays a message to the user. It is created using the **alert** keyword, followed by parentheses containing the message text.

```
alert("Thanks for your input!");
```

Key Rules

- **alert** must be lowercase — capitalizing it (e.g. Alert) breaks the script.
- Parentheses () are required immediately after the keyword — no space before them.
- Text inside quotes is called a **string** — a series of characters.
- JavaScript ignores spaces (outside of strings), but the style convention is to omit spaces around parentheses.
- Every statement should end with a **semicolon (;)** — except statements ending in a curly bracket.
- Use **double quotation marks** for strings (training convention), not single quotes.

Coding Alternatives to Know

- **window.alert(...)** is the formal/full way to write it — functionally identical to **alert(...)**, but the short form is preferred.
- Some coders use single quotes instead of double — legal as long as they match, but this training uses double quotes.

Quick takeaway: alert("message"); — lowercase keyword, no spaces around parens, double quotes, semicolon at the end.

Chapter 2: Variables for Strings

What is a Variable?

A variable is a name that refers to a value — like how "my name" refers to "Mark" in everyday language. JavaScript can substitute the value whenever the variable name is used.

Declaring and Assigning

```
var name = "Mark";
```

This single statement does two things: it **declares** the variable using **var**, and **assigns** the string "Mark" to it.

Reassigning a Value

```
var name = "Mark";  
name = "Ace";
```

Once a variable is declared with **var**, it does not need to be declared again. Later assignments simply use **variableName = newValue**; without the **var** keyword. A variable can hold only one value at a time, but that value can change.

Declare First, Assign Later

```
var nationality;  
nationality = "U.S.";
```

A variable can be declared without a value (left undefined), then assigned later in a separate statement. Any amount of code can sit between declaration and assignment.

Naming Has No Meaning to JavaScript

JavaScript does not care what a variable is named — **var floogle = "Mark"**; works exactly like **var name = "Mark"**;. However, for readability, variables should still be given **meaningful names**.

Variables vs. Strings — The Core Distinction

Variable	Text String
No quotes around it	Always enclosed in quotes
<pre>var lastName = "Smith"; (lastName is the variable)</pre>	"Smith" is the string

Using a Variable Inside an Alert

```
var thanx = "Thanks for your input!";  
alert(thanx);
```

JavaScript substitutes the value the variable refers to, so this displays the same message as `alert("Thanks for your input!");`

Quick takeaway: `var` creates a variable; `=` assigns a value; reassignment drops the `var` keyword; variable names are never quoted.

Chapter 3: Variables for Numbers

Assigning Numbers

```
var weight = 150;
```

Numbers are **not** enclosed in quotes. This is how JavaScript tells a number apart from a string. Because it's unquoted, JavaScript also confirms it isn't a variable name (a name can't start with a digit), so it must be a number.

Doing Math with Variables

```
var originalNum = 23;  
var newNum = originalNum + 7; // newNum becomes 30
```

Math expressions can mix variables and literal numbers, or use only variables, or only numbers — JavaScript treats them the same.

A Variable Can Use Itself

```
var originalNum = 90;  
originalNum = originalNum + 10; // now 100
```

The Quotes Trap: Strings vs. Numbers

```
var originalNum = "23";  
var newNum = originalNum + 7; // does NOT equal 30!
```

Putting quotes around a number turns it into a **string**. JavaScript cannot do normal addition on a string the way it does on a number — the result will not be a simple numeric sum.

Variable Naming with Numbers

- `var 1stPresident = "Washington";` — **illegal** (name starts with a digit).
- `var prezWhoCame1st = "Washington";` — **legal** (digit appears later in the name).

Numbers in Alerts

```
alert(144);  
var caseQty = 144;  
alert(caseQty);
```

JavaScript automatically converts a number to a string when displaying it in an alert box.

Quick takeaway: Numbers have no quotes. "23" + 7 is NOT 30 because "23" is a string, not a number.

Chapter 4: Variable Names — Legal and Illegal

Naming Rules

- Cannot be enclosed in quotation marks.
- Cannot be a number, or start with a number.

- Cannot be one of JavaScript's reserved keywords (e.g. alert, var) — though a name **can contain** a keyword, like userAlert or myVar.
- Cannot contain spaces.
- Can contain only letters, numbers, dollar signs (\$), and underscores (_).
- Are **case sensitive** — rose and Rose are two different variables.

camelCase Convention

Variable names should follow **camelCase**: start with a lowercase letter, and capitalize the first letter of each subsequent word (creating a "hump").

```
userResponse
userResponseTime
userResponseTimeLimit
response // single word, no hump needed
```

Naming Style: Balance Clarity and Brevity

Use **descriptive** names so code is understandable later (userName is better than x). But avoid going too far — bestSupportingActressInADramaOrComedy is clear but unwieldy and should be shortened.

Quick takeaway: letters, numbers, \$ and _ only; never start with a number; case sensitive; use camelCase.

Chapter 5: Math Expressions — Familiar Operators

Using Expressions Instead of Plain Numbers

```
var popularNumber = 2 + 2; // same as writing 4 directly
alert(2 + 2); // displays "4"
```

Anywhere a number can be used, a math expression can be used instead. JavaScript always evaluates the expression and uses the result.

The Standard Operators

Operator	Meaning	Example
+	Addition	2 + 2 → 4
-	Subtraction	12 - 24 → -12
*	Multiplication	3 * 12 → 36
/	Division	(standard division)
%	Modulus (remainder)	10 % 3 → 1

The Modulus Operator (%) Explained

```
var whatsLeftOver = 10 % 3; // 1 (remainder of 10 / 3)
var whatsLeftOver = 9 % 3; // 0 (divides evenly)
```

% gives the **remainder** of a division, not the quotient. If one number divides evenly into another, the result is 0.

Mixing Variables and Literals

```
var num = 10;  
var popularNumber = num + 200; // 210  
  
var num = 10;  
var anotherNum = 1;  
var popularNumber = num + anotherNum; // 11
```

Expressions can combine variables with literal numbers, or use only variables — JavaScript evaluates both the same way.

Quick takeaway: + - * / are standard math; % returns the remainder, not the quotient.

Next: practice exercises at ASmarterWayToLearn.com/js/ for each chapter (1-5), then move on to Chapter 6 onward.